```
(compared using ==)
    « backtrace truncated »
```

"Expected 200; got 400" is technically correct, but it does not provide enough information to understand *why* we got a 400 "Bad Request" response. The HTTP server is telling us what we did wrong in the response body, but that information is not included in the failure message. To include this extra information, you can pass an alternate failure message along with the matcher to `to` or `not_to`:

```
>> expect(resp.status).to eq(200), "Got a #{resp.status}: #{resp.body}"
RSpec::Expectations::ExpectationNotMetError: Got a 400: unknown query param ↩
`sort`
    « backtrace truncated »
```

If the failure message is expensive to generate (for example, scanning a large core dump file), you can pass in a callable object such as a `Proc` or a `Method` object. That way, you only pay the cost if the spec fails:

```
>> expect(resp.status).to eq(200), resp.method(:body)
RSpec::Expectations::ExpectationNotMetError: unknown query param `sort`
    « backtrace truncated »
```

Clear Failure Messages Save You Time



We can't tell you how many times we've seen "failed assertion, no message given" in legacy test suites. When you encounter a vague failure message like that, the best you can do is add a bunch of `puts` calls with debugging info and then rerun your tests.

A good error message tells you exactly what went wrong so that you can start fixing it right away. Time saved diagnosing failures can translate directly to lower project costs.

When the matcher's default failure message doesn't provide enough detail, a custom message may be just what you need. If you find yourself using the same message repeatedly, you can save time by writing your own matcher instead. We'll show you how to do that in *Creating Custom Matchers*.

RSpec Expectations vs. Traditional Assertions

It might feel like RSpec's expectations have a lot of moving parts. If you've used traditional assertions in an xUnit testing framework such as Ruby's built-in Minitest, you may be wondering if the added complexity is worth it. In fact, expectations and assertions are same basic concept, just with different emphases.